

Z Algorithm

The **Z Algorithm** is a string pattern-matching algorithm used to find a **pattern inside a text** efficiently.

It is similar to KMP, but the way it stores information is different.

Main Idea

For every position i , the Z Algorithm calculates:

How many characters starting from position i are the same as the characters starting from the beginning of the string?

This value is stored in an array called the **Z array**.

1. Simple Example

```
String = "AABAACAABAA"
```

```
Index : 0 1 2 3 4 5 6 7 8 9 10  
String: A A B A A C A A B A A
```

What does $Z[i]$ mean?

At index 1 :

```
Z[1] = 1
```

because only the first character matches the string prefix.

2. Building the Z Array

```
Z = [0, 1, 0, 1, 1, 0, 5, 1, 0, 2, 1]
```

3. Z-Box Concept

The Z Algorithm uses a special range called a **Z-box**.

- **L** = left boundary
- **R** = right boundary
- **S[L:R+1]** matches the prefix of the string

Example:

```
A A B A A C A A B A A
0 1 2 3 4 5 6 7 8 9 10
          L          R
          6          10
```

`Z[6] = 5`

4. Another Simple Example

`S = "AAAAA"`

`Z = [0, 4, 3, 2, 1]`

5. Pattern Matching

`Text = "ABABDABACDABABCABAB"`

`Pattern = "ABABCABAB"`

Create:

`Pattern + "$" + Text`

`ABABCABAB$ABABDABACDABABCABAB`

Whenever:

```
z[i] == len(pattern)
```

The pattern is found.

6. Python Implementation

```
def z_algorithm(s):
    n = len(s)
    z = [0] * n

    L = 0
    R = 0

    for i in range(1, n):
        if i <= R:
            z[i] = min(R - i + 1, z[i - L])

            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1

            if i + z[i] - 1 > R:
                L = i
                R = i + z[i] - 1

    return z
```

Output

```
String: AABAACAABAA
Z array: [0, 1, 0, 1, 1, 0, 5, 1, 0, 2, 1]
```

7. Pattern Search Program

```
def search(pattern, text):  
    combined = pattern + "$" + text  
    z = z_algorithm(combined)  
  
    pattern_length = len(pattern)  
  
    for i in range(len(z)):  
        if z[i] == pattern_length:  
            position = i - pattern_length - 1  
            print("Pattern found at index:", position)
```

Output:

```
Pattern found at index: 10
```

8. Time Complexity

Z Array Construction

$O(n)$

Pattern Matching

$O(m + n)$

Space Complexity

$O(m + n)$

9. Z vs KMP vs Rabin-Karp

Algorithm	Main Idea
KMP	Uses LPS array

Algorithm	Main Idea
Rabin-Karp	Uses hash values
Z Algorithm	Uses Z array

Easy Memory Trick

KMP → LPS
Rabin-Karp → Hash
Z Algorithm → Z Array

Z[i] = Number of characters from position *i* that match the beginning of the string.

This is the most important concept to remember.